SENSORS

Arduino_MKRIoTCarrier

GNU Lesser General Public License v2.1 V2.1.0

Riccardo Rizzo, Jose García, Pablo Marquínez 04/04/2024

Arduino <info@arduino.cc> https://github.com/arduino-li... info@arduino.cc

Controlling the IoT MKR Carrier
Allows you to control all the components included in the Explore IoT Kit

GO TO REPOSITORY

This is a basic documentation about the library for the *MKR IoT Carrier* .

The library can be downloaded from the Arduino IDE's library manager or by going to the **github repository**

This carrier has a lot of features and sensors to play with, some of them are the capacitive buttons, 5 RGB LEDs, the 240x240 RGB display and much more!

Its included in some kits:

- Explore IoT Kit
- Oplà IoT Kit

and Standalone:

- MKR IoT Carrier

Examples Included

- Actuators
  - Buzzer_Melody
  - Relays_blink
- All Features
- Display
  - Compose_Images
  - Graphics
  - Show_GIF (needs external library)
- LEDs
- SD_card
- Sensors
  - Environment
  - IMU
  - Light
  - Pressure
- TouchPads
  - CustomSensitivity
  - getTouch
  - Relays_control_Qtouch
  - Touch_Signals
  - Touch_and_LEDs
  - TouchTypes
- Interruptions (TODO)
- Arduino Cloud (TODO)

Classes

MKRIoTCarrier

Constructor of the object

```
MKRIoTCarrier yourName; //In the examples we name it as *carrier*
```

Initialization example sketch

```
#include <Arduino_MKRIoTCarrier.h>
MKRIoTCarrier carrier;

setup(){
Serial.begin(9600);
if(!carrier.begin(){  //It will see any sensor failure
   Serial.println("Failure on init");
   while(1);
 }
}
```

SD CARD

This can be accessed using the SD library commands that is already #included inside the library

The SD class initialized in the main `begin()`

The chip select (CS) pin can be known with SD_CS

Syntax Example

```
#include <Arduino_MKRIoTCarrier.h>
MKRIoTCarrier carrier;

File myFile;

setup(){
   carrier.begin();  //SD card initialized here

   myFile = SD.open("test.txt", FILE_WRITE);
}
```

Buttons class

Init the calaibration and the set up for the touchable pads (Already done in the MKRIoTCarrier class' begin())

```
Buttons.begin()
```

Read the state of the pads and save them to be analyze in the different type of touch events

```
Buttons.update()
```

Use TOUCHX being X a number from 0 to 4 (Button00 - Button04)

Get if the pad is getting touched, true until it gets released

```
Buttons.getTouch(TOUCHX)
```

Get when have been a touch down

```
Buttons.onTouchDown(TOUCHX)
```

Get when the button has been released

```
Buttons.onTouchUp(TOUCHX)
```

Get both, touched and released

```
Buttons.onTouchChange(TOUCHX)
```

In case you have another enclosure you can change the sensitivity of the pads, 3-100 Automatically configured when you call `carrier.withCase()`, by default is false (sensitivity threshold 4)

```
Buttons.updateConfig(int newSens)
```

Syntax example

```
#include <Arduino_MKRIoTCarrier.h>
MKRIoTCarrier carrier;

void setup(){
   Serial.begin(9600);
   carrier.begin();
}

void loop(){
   carrier.Buttons.update();    // Read the buttons state


   //Check if the Button 0 is being touched

   if (carrier.Buttons.getTouch(TOUCH0)){

      Serial.println("Touching Button 0");

   }



   //You can replace getTouch(), with the other types of touch events

   //If you use more than one type of events in the same Button its not going to be stable, watch out
   //In case you need it, after each touch event make a Buttons.update() to read it correctly
}
```

Buzzer class

Equivalent to tone(), it will make the tone with the selected frequency

```
Buzzer.sound(freq)
```

Equivalent to noTone(), it will stop the tone signal

```
Buzzer.noSound()
```

To emit a beep you can use the `beep()` method:

```
Buzzer.beep();
```

Relay class

Control both relays and get the status of them

You can control them by using `Relay1` and `Relay2`

Swap to the NormallyOpen (NO) circuit of the relay

```
RelayX.open()
```

Swap to the NormallyClosed (NC) circuit of the relay, default mode on power off

```
RelayX.close()
```

Bool, returns the status LOW means NC and HIGH means NO

```
RelayX.getStatus()
```

DotStar LEDs

It is controlled with the Adafruit's DotStar library

The documentation form Adafruit **here**

Syntax Example

```
#include <Arduino_MKRIoTCarrier.h>
MKRIoTCarrier carrier;

uint32_t myCustomColor = carrier.leds.Color(255,100,50);

void setup(){
   carrier.begin();
   carrier.leds.fill(myCustomColor, 0, 5);
   carrier.leds.show();
}

void loop(){}
```

Some examples Init the LEDs strip, done by default using the main begin()

```
leds.begin()
```

Sets the color of the index's LED

```
leds.setPixelColor(index, red, green, blue)
```

In case you have custom colors you can use this method too

```
leds.setPixelColor(index, color)
```

Set the overall brightness

```
leds.setBrightness(0-255)
```

Update the LEDs with the new values

```
leds.show()
```

Clear the buffer of the LEDs

```
leds.clear()
```

Fill X amount of the LEDs with the same color

```
leds.fill(color, firstLedToCount, count)
```

Save your custom color:

```
uint32_t myColor = carrier.leds.Color(red, green, blue)
```

Pressure sensor - LPS22HB (Barometric)

Visit our reference about LPS22HB

Syntax Example

```
float pressure;
pressure = carrier.Pressure.readPressure()
```

IMU - LSM6DS3

Visit our reference about LSM6DS3

Syntax Example

```
float aX,aY,aZ;
carrier.IMUmodule.readAcceleration(aX, aY, aZ);
```

Humidity and Temperature - HTS221

Visit our reference about HTS221

Syntax Example

```
float humidity;
humidity = carrier.Env.readHumidity();
```

Ambient light, Gesture and Proximity - APDS9960

Visit our reference about APDS9960

Syntax Example

```
int proximity;
proximity = carrier.Light.readProximity();
```

Display

It is controlled through the Adafruit-ST7735-Library

The resolution is 240 x 240.

Initialized inside the main `begin()`

Colors : every color with the prefix 'ST77XX_' i.e. ST77XX_BLACK. Colors available from the library are BLACK, WHITE, RED, GREEN, BLUE, CYAN, MAGENTA, YELLOW, ORANGE

In order to turn on the screen you will need to add this inside the setup() (already inside the main begin())

```
display.init(240, 240);
pinMode(TFT_BLACKLIGHT, OUTPUT);
digitalWrite(TFT_BLACKLIGHT,HIGH);
```

Colors : every color with the prefix 'ST77XX_' i.e. ST77XX_BLACK. Colors available from the library are BLACK, WHITE, RED, GREEN, BLUE, CYAN, MAGENTA, YELLOW, ORANGE

General:

Fill the entire screen with the selected color

```
display.fillScreen(color)
```

Get width and height

```
display.width() / height()
```

Rotates the coordinate system (number between 0 and 3)

```
display.setRotation(0-3)
```

Text:

Set the cursor to write text in the selected pixels

```
display.setCursor(screenX, screenY)
```

```
display.print(text)
```

It will print the string inside in the current cursor position

```
display.setTextColor(color)
```

Saves the selected color to print the text until the color is changed again

```
display.setTextSize(size)
```

Sets the size of the text that is going to be printed

```
display.setTextWrap(True/False)
```

Set the auto wrap of the text, if it is not the text will not jump to the next line.

Drawings:

Draw a Line from the start Vector to the End vector with the selected color, Use drawFastVLine() and drawFastHLine() introducing the same settings, to avoid the agular calc.

```
display.drawLine(startX, startY, endX, endY, color)
```

Draw a Circle from the center Vector with the selected radius and color

```
display.drawCircle(centerX, centerY, radius, color)
```

Draw a rectangle

```
display.drawRect(topLeftX, topLeftY, width, height, color)
```

Draw a filled rectangle

```
display.fillRect(topLeftX, topLeftY, width, height, color)
```

Draw a filled circle from the center Vector, with the selected radius and color

```
display.fillCircle(centerX, centerY, radius, color)
```

Draw a rounded rectangle

```
display.drawRoundRect(topLeftX, topLeftY, width, height, curveRadius, color)
```

Draw a filled and rounded rectangle

```
display.fillRoundRect(topLeftX, topLeftY, width, height, curveRadius, color)
```

Draw a triangle by introducing the 3 points and color

```
display.drawTriangle(x0, y0, x1, y1, x2, y2, color)
```

Draw a filled triangle

```
display.fillTriangle(x0, y0, x1, y1, x2, y2, color)
```

Draw a character

```
display.drawChar(topLeftX, topLeftY, character, color, backgroundColor, size)
```

Draw a bitmap, the bitmap needs to be with PROGMEM

```
display.drawBitmap(startX, startY, bitmap, width, height, color)
```